

CDT4 - LECTURE SUMMARY

CDT4 Topics Covered	Performance analysis: Asymptotic notations omega and theta
Motivation (Why you (students) should learn these topics?)	(As course faculty, how do you motivate your course students to learn these topics? Students will learn the effective analysis Asymptotic notation of an algorithm to define the mathematical foundation/framing of its run-time performance. Using asymptotic analysis, we can very well conclude the best case, average case, and worst case scenario of an algorithm.
Lecture Learning Outcomes (LLOs): After completion of this lecture, you should be able to...	
LLO1 On topic 1	<i>compute the time complexities for any type of notations</i>
LLO2 On topic 2	<i>compute the space complexities for any type of notations</i>

CDT4 – Lecture Summary – Key Takeaways

Asymptotic analysis of an algorithm refers to defining the mathematical boundation/framing of its run-time performance. Using asymptotic analysis, we can very well conclude the best case, average case, and worst case scenario of an algorithm.

Asymptotic analysis is input bound i.e., if there's no input to the algorithm, it is concluded to work in a constant time. Other than the "input" all other factors are considered constant.

Asymptotic analysis refers to computing the running time of any operation in mathematical units of computation. For example, the running time of one operation is computed as $f(n)$ and may be for another operation it is computed as $g(n^2)$. This means the first operation running time will increase linearly with the increase in n and the running time of the second operation will increase exponentially when n increases. Similarly, the running time of both operations will be nearly the same if n is significantly small.

Usually, the time required by an algorithm falls under three types –

- **Best Case** – Minimum time required for program execution.
- **Average Case** – Average time required for program execution.
- **Worst Case** – Maximum time required for program execution.

Asymptotic Notations

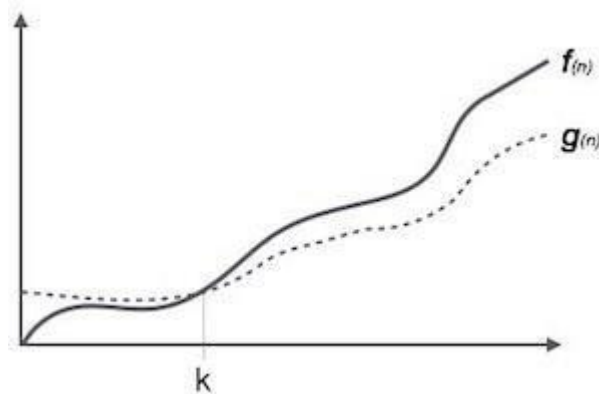
Following are the commonly used asymptotic notations to calculate the running time complexity of an algorithm.

- O Notation
- Ω Notation
- θ Notation

Omega Notation, Ω

The notation $\Omega(n)$ is the formal way to express the lower bound of an algorithm's running time. It measures the best case time complexity or the best amount of time an algorithm can possibly take to complete.

CDT4 - LECTURE SUMMARY



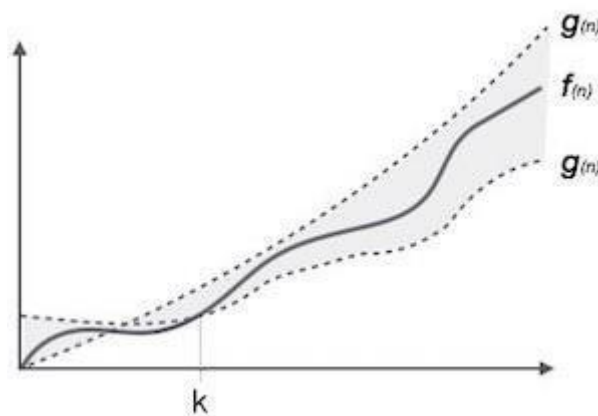
For example, for a function $f(n)$

$\Omega(f(n)) \geq \{ g(n) : \text{there exists } c > 0 \text{ and } n_0 \text{ such that } g(n) \leq c \cdot f(n) \text{ for all } n > n_0. \}$

Theta Notation, θ

The notation $\theta(n)$ is the formal way to express both the lower bound and the upper bound of an algorithm's running time. It is represented as follows –

$\theta(f(n)) = \{ g(n) \text{ if and only if } g(n) = O(f(n)) \text{ and } g(n) = \Omega(f(n)) \text{ for all } n > n_0. \}$



Common Asymptotic Notations

Following is a list of some common asymptotic notations –

constant	–	$O(1)$
logarithmic	–	$O(\log n)$
linear	–	$O(n)$
$n \log n$	–	$O(n \log n)$
quadratic	–	$O(n^2)$
cubic	–	$O(n^3)$

CDT4 - LECTURE SUMMARY

polynomial	—	$n^{O(1)}$
exponential	—	$2^{O(n)}$

CDT4 - LECTURE LEVEL PRACTICE PROBLEMS (LLPs) to test the LLOs

To test whether you achieved the learning outcomes of this lecture, you should be able to solve the following LLPs, in the class itself, after completion of the lecture. Minimum one question / problem (LLP) is designed to test each expected LLO.

LLP1(on LLO1): Compute the Big 'O' notations for the following

$$F(n)=5n^2+2n+3$$

$$F(n)=100n^3+50n+5$$

LLP2 (on LLO2): Compute the small 'o' notations for the following

$$F(n)=n^2$$